我计划使用VQMC方法来计算H2分子系统的基态和激发态能量。 H2分子在二次量子化下、满足自旋约束、粒子数约束的情况下，可以被视为有四个合法组态：

Hilbert空间信息: 空间轨道数: 2 自旋轨道数: 4 电子数: 2 (α=1, β=1) Hilbert空间维度: 4 所有可能的电子组态: [[0 1 0 1] [0 1 1 0] [1 0 0 1] [1 0 1 0]] #组态顺序是：[α0 α1 β0 β1]

我使用 FFNN 作为 VQMC 方法的波函数Ansatz:

```
import netket as nk
import netket.experimental as nkx
import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf, fci
import jax
import jax.numpy as jnp
from flax import nnx
#==============================================================================
# 1. 分子定义 + FCI 精确基准（你的代码）
#==============================================================================
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]

mol = gto.M(atom=geometry, basis='STO-3G')
mf = scf.RHF(mol).run(verbose=0)
E_hf = mf.e_tot

# FCI 精确解（4个态）
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()

print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0])*27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# 转为 NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

#==============================================================================
# 2. Hilbert 空间 + 采样器（你的代码）
#==============================================================================
n_orbitals = 2
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=n_orbitals,
    s=1/2,
    n_fermions_per_spin=(1, 1)
)

g = nk.graph.Graph(edges=[(0,1), (2,3)])
sampler = nk.sampler.MetropolisFermionHop(
    hi, graph=g, n_chains=4, spin_symmetric=True, sweep_size=64
)
```

在 NES-VMC 的文章中指出: $\psi(x_1)$ 中 ψ是指的 FFNN 波函数 x_1 指的是某一个组态例如[1,0,1,0]. 此外 $\hat{H}\psi(x_1)$ 是 H2 分子系统的哈密顿量$\hat{H}$作用在 ψ 之后

代入 x_1 得到的结果。

```
import netket as nk
import netket.experimental as nkx
import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf, fci
import jax
import jax.numpy as jnp
from flax import nnx


#==============================================================================
# 1. 分子定义 + FCI 精确基准（你的代码）
#==============================================================================
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]

mol = gto.M(atom=geometry, basis='STO-3G')
mf = scf.RHF(mol).run(verbose=0)
E_hf = mf.e_tot

# FCI 精确解（4个态）
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()

print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0])*27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# 转为 NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

#==============================================================================
# 2. Hilbert 空间 + 采样器（你的代码）
#==============================================================================
n_orbitals = 2
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=n_orbitals,
    s=1/2,
    n_fermions_per_spin=(1, 1)
)

g = nk.graph.Graph(edges=[(0,1), (2,3)])
```

以上是一些固定的基于 Netket 框架的问题描述 主要是针对H2 分子系统的。

```
#==============================================================================
# 3. 神经网络模型（你的代码完全不变）
#==============================================================================
class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
```

```
        h = nnx.tanh(self.linear1(x))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

class NESTotalAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals, n_states=2, hidden_dim=8, *, rngs: nnx.Rngs):
        super().__init__()
        self.n_states = n_states
        self.n_spin_orbitals = n_spin_orbitals
        self.single_ansatz_list = [
            SingleStateAnsatz(n_spin_orbitals, hidden_dim, rngs=nnx.Rngs(42+i))
            for i in range(n_states)
        ]

    def __call__(self, x_batch, return_log_psi=False):  # ✅ 加开关
        # 你的原有逻辑
        K = self.n_states
        M = []
        for i in range(K):
            row = [self.single_ansatz_list[j](x_batch[i]) for j in range(K)]
            M.append(jnp.array(row))
        M = jnp.stack(M)
        psi_total = jnp.linalg.det(M)
        log_psi_total = jnp.log(psi_total)  # NetKet 必须用 log

        # ✅ 关键：Sampler 只需要 log_psi（一个值）
        if return_log_psi:
            return log_psi_total
        # ✅ 算能量时返回两个值
        else:
            return psi_total, M
```

以上是针对 NES-VMC 算法中 对于 Total Ansatz 波函数的实现, Total Ansatz 根据定义应当是 Single Ansatz 的矩阵组合。

```
#==============================================================================
# 4. 核心：HΨ + 局部能量（你的代码 100% 不变）
#==============================================================================
def Ham_psi(ha: nk.operator.DiscreteOperator, model, x: jnp.array):
    x_primes, mels = ha.get_conn(x)
    psi_vals = jax.vmap(model)(x_primes)
    return jnp.sum(mels * psi_vals)

def Ham_Psi(ha, total_ansatz: NESTotalAnsatz, x_batch):
    K = total_ansatz.n_states
    H_mat = []
    for i in range(K):
        row = []
        for j in range(K):
            v = Ham_psi(ha, total_ansatz.single_ansatz_list[j], x_batch[i])
            row.append(v)
        H_mat.append(row)
    return jnp.array(H_mat, dtype=complex)

def compute_local_energy(ha, total_ansatz: NESTotalAnsatz, x_batch):
    psi, M = total_ansatz(x_batch)
    Hp = Ham_Psi(ha, total_ansatz, x_batch)
    el_mat = jnp.linalg.solve(M, Hp)
    return jnp.trace(el_mat), el_mat
```

```python
#==============================================================================
# 5. ✅ 论文标准：损失函数 + 自动梯度（JAX 正确求导）
#==============================================================================
@jax.jit
def loss_fn(model_state, samples):
    # 绑定参数
    nnx.update(total_ansatz, model_state)

    total_energy = 0.0 + 0j
    n_samples = samples.shape[0]

    for xb in samples:
        tr_EL, _ = compute_local_energy(ha, total_ansatz, xb)
        total_energy += tr_EL

    avg_energy = total_energy.real / n_samples
    return avg_energy

@jax.jit
def train_step(model_state, opt_state, samples):
    # 🔥 自动计算论文所需的参数梯度
    loss, grads = jax.value_and_grad(loss_fn)(model_state, samples)
    model_state = jax.tree_util.tree_map(lambda p, g: p - 0.01 * g, model_state, grads)
    return model_state, opt_state, loss
```

使用如下的方式设置采样器

```python
g = nk.graph.Graph(edges=[(0,1), (2,3)])
sampler = nk.sampler.MetropolisFermionHop(
    hi, graph=g, n_chains=2, spin_symmetric=True, sweep_size=64
)
parameters = nnx.split(total_ansatz)
sampler_state = sampler.init_state(forward, parameters, seed=1)
sampler_state = sampler.reset(forward, parameters, sampler_state)

samples, sampler_state = sampler.sample(
    forward, parameters, state=sampler_state, chain_length=100
)
```

如上的代码中 samples.shape=(2, 100, 4)，即每个链采样了 100 个样本，每个样本的形状为 (4, 1)。

其中关于 Netket里的采样器的相关 API： netket.sampler.MetropolisFermionHop netket.sampler.MetropolisFermionHop(hilbert, *, clusters=None, graph=None, d_max=1, spin_symmetric=True, dtype=<class 'numpy.int8'>, **kwargs)[source] This sampler moves (or hops) a random particle to a different but random empty mode. It works similar to MetropolisExchange, but only allows exchanges between occupied and unoccupied modes.

Parameters
hilbert – The Hilbert space to sample.

d_max – The maximum graph distance allowed for exchanges.

spin_symmetric – (default True) If True, exchanges are only allowed between modes with the same spin projection.

n_chains – The total number of independent Markov chains across all JAX devices. Either specify this or n_chains_per_rank.

n_chains_per_rank – Number of independent chains on every JAX device (default = 16).

sweep_size – Number of sweeps for each step along the chain. Defaults to the number of sites in the Hilbert space. This is equivalent to subsampling the Markov chain.

reset_chains – If True, resets the chain state when reset is called on every new sampling (default = False).

machine_pow – The power to which the machine should be exponentiated to generate the pdf (default = 2).

dtype – The dtype of the states sampled (default = np.int8).

Return type
MetropolisSampler

我希望 将 n_chains 设置为 K ， 其中 K 是NES-VMC 中的K个能态。

nnx.call方法：

flax.nnx.call(graphdef_state, /)[源代码] 调用由 (GraphDef, State) 对定义的底层图节点的方法。

call 接收一个 (GraphDef, State) 对，并创建一个代理对象，该对象可用于调用底层图节点上的方法。当调用一个方法时，其输出将与代表图节点更新后状态的新的 (GraphDef, State) 对一起返回。call 等效于 merge() > method > split`()，但在纯 JAX 函数中使用起来更方便。

示例

from flax import nnx import jax import jax.numpy as jnp

class StatefulLinear(nnx.Module): def **init**(self, din, dout, rngs): self.w = nnx.Param(jax.random.uniform(rngs(), (din, dout))) self.b = nnx.Param(jnp.zeros((dout,))) self.count = Variable(jnp.array(0, dtype=jnp.uint32))

def increment(self): self.count += 1

def **call**(self, x): self.increment() return x @ self.w + self.b

linear = StatefulLinear(3, 2, nnx.Rngs(0)) linear_state = nnx.split(linear)

@jax.jit def forward(x, linear_state): y, linear_state = nnx.call(linear_state)(x) return y, linear_state

x = jnp.ones((1, 3)) y, linear_state = forward(x, linear_state) y, linear_state = forward(x, linear_state)

linear = nnx.merge(*linear_state) linear.count.value Array(2, dtype=uint32)

$x^1,x^2,x^3,..,x^K$ 服从 $\Psi^2(x^1,x^1,x^3,..,x^k)$

[redacted]

[redacted]